Data cleaning

Introduction

The aim of this project is to collect data, identify anomalies and, finally, clean the data. The tools used are Python and Jupyter Notebook.

1. **Collect data**

This involves collecting data, checking the data type and visualising the data

```
[1]: #import module
     import numpy as np
     import pandas as pd
     import matplotlib.pyplot as plt
```

```
[3]: #collect data
     df= pd.read_csv("/home/djodj/Downloads/laptop_pricing.csv")
```

```
[5]: df.head()
```

[5]:

	Unnamed: 0	Manufacturer	Category	Screen	GPU	OS	CPU_core	Screen_Size_cm	CPU_frequency	RAM_GB	Storage_GB_SSD	Weight_kg	Price
0	0	Acer	4	IPS Panel	2	1	5	35.560	1.6	8	256	1.60	978
1	1	Dell	3	Full HD	1	1	3	39.624	2.0	4	256	2.20	634
2	2	Dell	3	Full HD	1	1	7	39.624	2.7	8	256	2.20	946
3	3	Dell	4	IPS Panel	2	1	5	33.782	1.6	8	128	1.22	1244
4	4	HP	4	Full HD	2	1	7	39.624	1.8	8	256	1.91	837

```
[6]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 238 entries, 0 to 237
Data columns (total 13 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   Unnamed: 0      238 non-null    int64
 1   Manufacturer    238 non-null    object
 2   Category        238 non-null    int64
 3   Screen          238 non-null    object
 4   GPU             238 non-null    int64
 5   OS              238 non-null    int64
 6   CPU_core        238 non-null    int64
 7   Screen_Size_cm  234 non-null    float64
 8   CPU_frequency   238 non-null    float64
 9   RAM_GB          238 non-null    int64
 10  Storage_GB_SSD  238 non-null    int64
 11  Weight_kg       233 non-null    float64
 12  Price           238 non-null    int64
dtypes: float64(3), int64(8), object(2)
memory usage: 24.3+ KB
```

As the decimal data has two digits after the decimal point, let’s set the Screen_Size_cm column to two digits after the decimal point.

```
[7]: df[['Screen_Size_cm']] = np.round(df[['Screen_Size_cm']],2)
     df.head()
```

[7]:

	Unnamed: 0	Manufacturer	Category	Screen	GPU	OS	CPU_core	Screen_Size_cm	CPU_frequency	RAM_GB	Storage_GB_SSD	Weight_kg	Price
0	0	Acer	4	IPS Panel	2	1	5	35.56	1.6	8	256	1.60	978
1	1	Dell	3	Full HD	1	1	3	39.62	2.0	4	256	2.20	634
2	2	Dell	3	Full HD	1	1	7	39.62	2.7	8	256	2.20	946
3	3	Dell	4	IPS Panel	2	1	5	33.78	1.6	8	128	1.22	1244
4	4	HP	4	Full HD	2	1	7	39.62	1.8	8	256	1.91	837

```
[ ]:
```

2. **Data cleaning**

➔ Detection of missing values

Checking whether our data contains any missing values

```
[8]: df.isnull().sum()
```

```
[8]: Unnamed: 0        0
     Manufacturer      0
     Category          0
     Screen            0
     GPU               0
     OS                0
     CPU_core          0
     Screen_Size_cm    4
     CPU_frequency     0
     RAM_GB            0
     Storage_GB_SSD    0
     Weight_kg         5
     Price             0
     dtype: int64
```

The dataframe contains missing values in the Screen_size_cm and Weight_kg columns. So let's handle the missing data.

➔ Handling missing data

In the Screen_size_cm column, there is a duplicate value for “size,” so the missing data will be replaced with the most frequent value.

```
c_size =  df['Screen_Size_cm'].value_counts().idxmax()
df["Screen_Size_cm"].replace(np.nan, c_size, inplace=True)
```

For the Screen_size column, missing data is replaced with the average

```
weigh_avg = df['Weight_kg'].astype('float').mean(axis=0)
df['Weight_kg'] = df['Weight_kg'].replace(np.nan, weigh_avg, inplace = True)
```

```
/tmp/ipykernel_27299/2700665286.py:2: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.


  df['Weight_kg'] = df['Weight_kg'].replace(np.nan, weigh_avg, inplace = True)
```

➔ data type

These data types are appropriate for our situation

➔ Standardization

The value for Screen_size is typically expressed in inches. Similarly, the weight of a laptop should be given in pounds. So let’s perform the conversions.

```
# 1 inch = 2.54 cm and 1 kg = 2.205 pounds
df["Screen_Size_cm"] = df["Screen_Size_cm"]/2.54
df["Weight_kg"] = df["Weight_kg"]* 2.205
df.rename(columns={"Screen_Size_cm":"Screen_Size_inch", "Weight_kg":"Weight_pounds"}, inplace = True)
```

➔ Normalisation

Let's normalize the CPU_frequence column

```
df["CPU_frequency"] = df["CPU_frequency"]/df["CPU_frequency"].max()
```

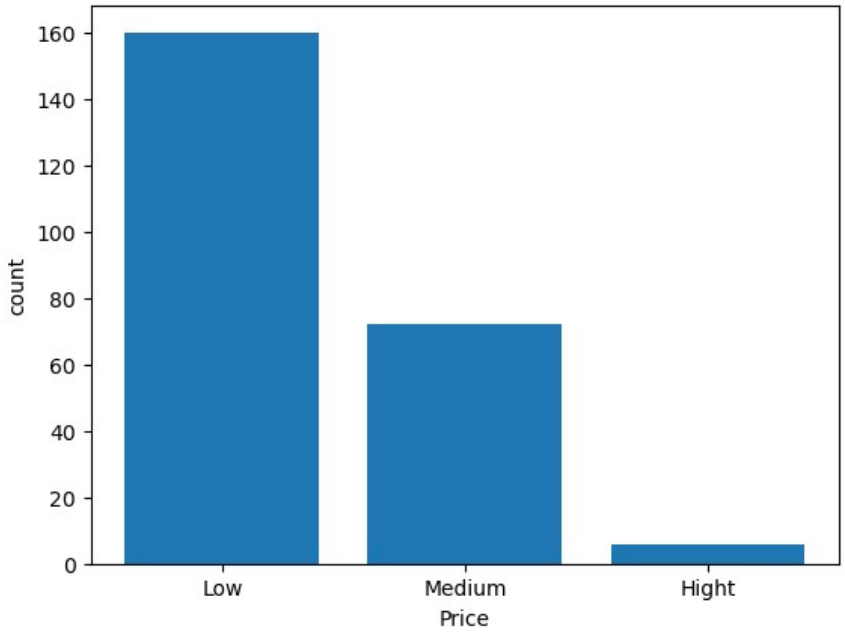
➔ Binning

Let’s group the “price” attribute into three classes and view the histogram

```
bins = np.linspace(min(df["Price"]), max(df["Price"]), 4)
group_name = ["Low", "Medium", "Hight"]
df["Price_binned"] = pd.cut(df["Price"], bins, labels=group_name, include_lowest=True)
```

```
plt.bar(group_name, df["Price_binned"].value_counts())
plt.xlabel("Price")
plt.ylabel("count")
```

```
Text(0, 0.5, 'count')
```



Conclusion

Using Jupyter Notebook and Python modules, we were able to clean the data to ensure we had high-quality data for analysis and sound decision-making.